

Play Pulse Studios Online Game Data Analysis

COMP1013

Jodie Nguyen-Hoang

2026-05-31

22158896

Declaration

By including this statement, we the authors of this work, verify that:

- We hold a copy of this assignment that we can produce if the original is lost or damaged.
- We hereby certify that no part of this assignment/product has been copied from any other student's work or from any other source except where due acknowledgement is made in the assignment.
- No part of this assignment/product has been written/produced for us by another person except where such collaboration has been authorised by the subject lecturer/tutor concerned.
- We are aware that this work may be reproduced and submitted to plagiarism detection software programs for the purpose of detecting possible plagiarism (which may retain a copy on its database for future plagiarism checking).
- We hereby certify that we have read and understand what the School of Computing, Engineering and Mathematics defines as minor and substantial breaches of misconduct as outlined in the learning guide for this unit.

1 GitHub

1.1 Version Control

This project uses Git for version control to track changes and manage progress throughout the Play Pulse Studios Online Game Data Analysis. The following commands were used to manage the repository during the project.

1.1.1 Basic Commands

These commands were used to stage, commit and push files to GitHub.

```
# Check whether files have been changed, added or deleted
git status
# Add all changed files to be committed
git add .
# Save work with a description
git commit -m "Add R Markdown project files"
# Upload files to GitHub for the first time
git push -u origin main
# Upload files to GitHub after the first time
git push
```

1.1.2 Viewing History

These commands were used to view past commits and track changes throughout the project.

```
# View all past commits
git log
# View past commits in a one line format
git log --oneline
# View past commits with a table or plot
git log --oneline --graph
```

1.1.3 Undoing Changes

These commands were used to undo changes when mistakes were made during the analysis.

```
# Undo changes made to a file before it has been committed
# This restores the file back to the last saved version
git checkout -- Analytics-Programming-Project.Rmd
# Undo the last commit but keep the file changes
git reset --soft HEAD~1
# Undo the last commit and discard all file changes
git reset --hard HEAD~1
```

2 Player Engagement Analysis By Experience Level

This task analyses player behaviour across different experience levels on the Play Pulse Studios platform. Players are grouped into three categories based on their signup date, which reflects their experience level. Key metrics such as number of players, average play time, and average score are then compared across these groups to identify differences in engagement and performance.

2.0.1 Assumptions

It is assumed that signup date accurately reflects a player's experience level, with earlier sign-ups that indicate more experience. It is also assumed that all recorded sessions represent genuine gameplay. In addition, average play time and average score are used as handy measures of player engagement and performance across these groups.

2.1 Experience Group

In order to analyse player behaviour across different experience levels, we will run the code to group them into:

- Veteran: before 2023
- Intermediate: 2023
- New: after 2023

```
players = players %>%
  mutate(
    # Convert signup_date to a proper date format
    signup_date = as.Date(signup_date),
    # First ifelse() checks if player signed up before 2023 -> "Veteran"
    # If not, the second ifelse() checks if they signed up in 2023 -> "Intermediate"
    # If neither condition is met, it means they signed up after 2023 -> "New"
    experience_group = ifelse(year(signup_date) < 2023, "Veteran",
                              ifelse(year(signup_date) == 2023, "Intermediate",
                                    "New")))
    # Then is stored into the experience_group variable
```

2.2 Number of Players

Now to code the number of players in each experience group.

```
# Count the number of players in each experience group
num_players = players %>%
  # The code groups the rows by experience_group
  group_by(experience_group) %>%
  # Function n() counts the number of rows in each group, which gives us the
  # number of players per group
  summarise(number_of_players = n())
```

2.2.1 Using kableExtra To Tabulate The Number of Players Per Group

```
num_players %>%
  kbl(
    # Rename the columns
    col.names = c("Experience Group", "Number of Players"),
    # # Left align the first column and centre align the second
    align      = c("l", "c")
  ) %>%
  kable_styling(
    # "striped" to change row colours
    # "bordered" adds lines around every cell
    bootstrap_options = c("striped", "bordered"),
    full_width = FALSE, # table is only as wide as it needs to be
    position = "left" # aligns table to the left side of page
  ) %>%
  # Styles the header row and colour of words
  row_spec(0, bold = TRUE, background = "#333333", color = "white") %>%
  column_spec(2, color = "black")
```

Experience Group	Number of Players
Intermediate	1851
New	1363
Veteran	1786

2.2.2 Findings For Number of Players In Each Group

The data shows that across the three experience groups, the Intermediate group is the largest, with **1,851 players**. Veterans follow with **1,786 players**, while New players make up the smallest group at **1,363 players**.

Intermediate players account for the biggest portion of the player base, which suggests that 2023 was a particularly strong year for new sign-ups on the platform. Veterans come in as the second largest group, indicating a solid and stable community of long-term players who joined before 2023. Even though New players form the smallest group, their total of 1,363 players is still quite significant. This shows that the platform continues to attract fresh users even after 2023.

Taken together, these figures point to a healthy and steadily growing player base for Play Pulse Studios, supported by a strong core of experienced players and a consistent stream of new arrivals over time.

2.3 Average Play Time By Experience Group

To be able to calculate the average play time for each experience group, we must first code the `experience_group` variable into the `sessions` variable.

```
# The function left_join keeps all sessions and matches player info where available
# It is stored in the sessions_joined variable
sessions_joined = sessions %>%
  left_join(players %>% select(c(player_id, experience_group)), by = "player_id")
# cat() prints the value for sessions that could not be matched to a player (NA values)
```

```
cat("The NA values for sessions that matched to a player is",
    sum(is.na(sessions_joined$experience_group)))
```

```
## The NA values for sessions that matched to a player is 0
```

Now that we have determined the NA value is 0, we do not need to filter it out again.

```
avg_playtime_group = sessions_joined %>%
  # Groups by each experience group
  group_by(experience_group) %>%
  # Removes any NA values then calculate the mean of the play time minutes in each group
  summarise(avg_play_time_mins = mean(play_time_minutes, na.rm = TRUE),)
```

```
avg_playtime_group %>%
  kbl(
    col.names = c("Experience Group", "Average Play Time (min)",
    align = c("l", "c")) %>%
  kable_styling(
    bootstrap_options = c("striped", "bordered"),
    full_width = FALSE,
    position = "left") %>%
  row_spec(0, bold = TRUE, background = "#333333", color = "white") %>%
  column_spec(2, bold = TRUE, color = "black")
```

Experience Group	Average Play Time (min)
Intermediate	91.72935
New	92.41463
Veteran	92.26217

2.3.1 Plotting The Average Play Time By Experience Group

Now to code the average play time by experience group into a bar chart.

```
# Use avg_playtime_group to create a bar chart
ggplot(avg_playtime_group,
  aes(x = experience_group,
    y = avg_play_time_mins,
    fill = experience_group)) +

  # Draws the bars
  geom_col(width = 0.5) +

  # Labels and small adjustments
  geom_text(
    aes(label = paste0(avg_play_time_mins, " min")),
    vjust = -0.5,
    size = 4) +

  # Assigns colours to the bar of each experience group
  scale_fill_manual(values = c(
```

```

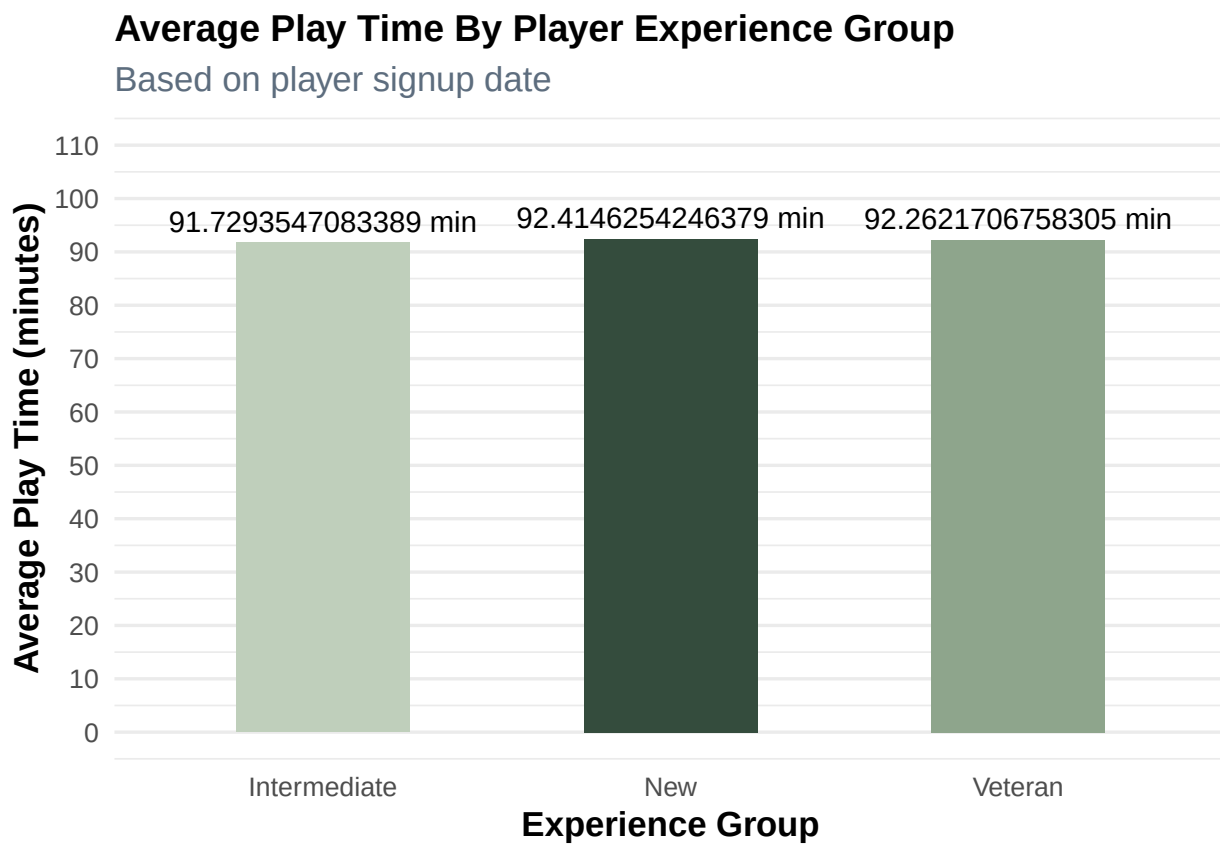
"Veteran" = "#8EA58C",
"Intermediate" = "#BFCFBB",
"New" = "#344C3D")) +

# Leaves space above bars for labels
scale_y_continuous(
  limits = c(0, 110),
  breaks = seq(0, 110, 10)) +

# Adds titles and axis labels
labs(
  title = "Average Play Time By Player Experience Group",
  subtitle = "Based on player signup date",
  x = "Experience Group",
  y = "Average Play Time (minutes)",) +

# Applies themes to make it look nice
theme_minimal(base_size = 13) +
theme(
  legend.position = "none",
  plot.title = element_text(face = "bold", size = 14),
  plot.subtitle = element_text(colour = "#5D6D7E"),
  axis.title = element_text(face = "bold"),
  panel.grid.major.x = element_blank())

```



2.3.2 Findings For Average Play Time In Each Group

The data shows that players spend nearly the same amount of time in each session, regardless of how long they have been on the platform.

New players tend to stay the longest, with an average session time of **92.41 minutes**. Veterans are very close behind at **92.26 minutes**, while Intermediate players average **91.73 minutes**. The difference between the highest and lowest groups is just **0.68 minutes**, which is extremely small. This level of consistency suggests that once players join the platform, their session length are not largely affected by their level of experience. Engagement appears to stay steady across all groups.

The slightly longer sessions among New players could be because they are still getting familiar with the platform. They are likely exploring different features and trying out various games. In contrast, Veterans may spend their time more efficiently since they already know what they enjoy and tend to focus on those activities.

2.4 Average Score of Each Experience Group

Using `sessions_joined` from earlier so we can calculate the average score per group.

```
avg_score_group = sessions_joined %>%
  # Filters out if there are any sessions that have NA in the experience_group variable
  # !is.na find rows where group is NOT missing
  filter(!is.na(experience_group)) %>%
  # Group by experience_group so the calculation is done per group
  group_by(experience_group) %>%
  # Removes any NA values then calculates the average score per group
  summarise(avg_score_per_group = mean(score, na.rm = TRUE))
```

2.4.1 Using kableExtra To Tabulate The Average Score Per Group

```
avg_score_group %>%
  kbl(
    # Rename the columns
    col.names = c("Experience Group", "Average Score"),
    # Left align the first column and centre align the second
    align      = c("l", "c")
  ) %>%
  kable_styling(
    # "striped" to change row colours
    # "bordered" adds lines around every cell
    bootstrap_options = c("striped", "bordered"),
    full_width = FALSE, # table only as wide as it needs to be
    position = "left" # align table to the left of the page
  ) %>%
  # Styles the header row and colour of words
  row_spec(0, bold = TRUE, background = "#333333", color = "white") %>%
  column_spec(2, bold = TRUE, color = "black")
```

Experience Group	Average Score
Intermediate	498.2044
New	495.9630
Veteran	501.0407

2.4.2 Findings For Average Scores Per Group

The data on average scores across the three experience groups shows that Veterans achieved the highest average score at **501.04**, followed by Intermediate players at **498.20**, and New players at **495.96**.

It is not surprising that Veterans come out on top, as their longer time on the platform gives them more opportunity to build strong skills and refine their strategies. With the most experience, they are more familiar with the games and can perform at a higher level. Intermediate players fall in the middle with an average score of **498.20**, which shows their moderate level of experience. They have had enough time to develop some skill but have not yet reached the same level of proficiency as Veterans. Whereas, New players recorded the lowest average score at **495.96**, which makes sense since they have only recently joined and are still learning the mechanics. However, the gap between New players and Veterans is just **5.08 points**, which is surprisingly small. This suggests that new players are picking things up quickly and adapting well to the games.

Looking at the results as a whole, the scores across all three groups are very close. This indicates that the games on the platform are accessible and enjoyable for players at all experience levels, allowing new users to stay competitive while still rewarding more experienced players.

3 Game Genre Analysis

This task examines player engagement across the five game genres (Action, Puzzle, RPG, Sports, and Strategy) on the Play Pulse Studios platform. It compares key metrics such as average play time, average score, number of sessions, and number of unique players to identify which genres are the most popular and engaging.

3.0.1 Assumptions

It is assumed that the genre classification reflects each game's type on the platform. A higher number of sessions and unique players is taken to consideration that it means greater popularity and engagement. Average play time and average score are also used as measures of engagement and performance. Since players can play multiple genres, the same player may be counted in more than one genre.

3.1 Average Play Time For Each Genre

To be able to calculate the average play time for each of the five genres, we must first code the `genre` variable into the `sessions` variable.

```
# The function left_join keeps all sessions and matches player info where available
# It is stored in the sessions_games variable
sessions_games = sessions %>%
  left_join(
    games %>% select(game_id, genre), by = "game_id")

# cat() prints the value for sessions that could not be matched to a game (NA values)
cat("The NA values for sessions that matched to a game is",
    sum(is.na(sessions_games$genre)))
```

```
## The NA values for sessions that matched to a game is 0
```

Now that we have determined the NA value is 0, we do not need to filter it out again.

```
avg_playtime_genre = sessions_games %>%
  # Groups by each genre
  group_by(genre) %>%
  # Removes any NA values then calculates the mean of the play time minutes in each genre
  summarise(avg_play_time_mins2 = mean(play_time_minutes, na.rm = TRUE))
```

3.1.1 Using kableExtra To Tabulate The Average Playtime In Each Genre

```
avg_playtime_genre %>%
  kbl(
    # Rename the columns
    col.names = c("Genre", "Average Play Time (min)"),
    # Left align the first column and centre align the second
    align = c("l", "c")
  ) %>%
```

```
kable_styling(
  # "striped" to change row colours
  # "bordered" adds lines around every cell
  bootstrap_options = c("striped", "bordered"),
  full_width = FALSE, # table only as wide as it needs to be
  position = "left" # align table to the left side of the page
) %>%
# Styles the header row and colour of words
row_spec(0, bold = TRUE, background = "#333333", color = "white") %>%
column_spec(2, bold = TRUE, color = "black")
```

Genre	Average Play Time (min)
Action	92.09816
Puzzle	91.67226
RPG	91.41153
Sports	92.58573
Strategy	92.88334

3.1.2 Findings For Average Play Time In Each Genre

The data of average play time across the five genres shows that session length is very consistent, ranging from **91.4 to 92.9 minutes**.

Strategy games have the highest average play time at **92.9 minutes**, likely due to the complexity of the game as it requires more time for planning and decision-making. Sports games follow at **92.6 minutes**, which may showcase the competitiveness of the game. Action games sit in the middle at **92.1 minutes**, showing steady engagement. RPG games record the lowest average at **91.4 minutes**, which could suggest players engage in shorter but more frequent sessions. Puzzle games are slightly higher at **91.7 minutes**, which fits perfectly with their short, task based gameplay.

Overall, there is only a **1.5 minutes** difference between genres. This suggests that session length remains consistent across all game types, which leads to a balanced level of engagement on the platform.

3.2 Average Score Per Genre

To calculate the average score for each genre, we will use the `sessions_games` variable to group the `genre` variable and then calculate the mean.

```
# Using sessions_games from earlier so we can calculate average score per genre
avg_score_genre = sessions_games %>%
  # Filters out if there are any sessions that have NA in the genre variable
  # !is.na find rows where genre is NOT missing
  filter(!is.na(genre)) %>%
  # The code groups by genre so the calculation is done per genre
  group_by(genre) %>%
  # Removes any NA values then calculates the average score per genre
  summarise(avg_score_per_genre = mean(score, na.rm = TRUE))
```

3.2.1 Using kableExtra To Tabulate The Average Score Per Genre

```
avg_score_genre %>%
  kbl(
    # Rename the columns
    col.names = c("Genre", "Average Score by Genre"),
    # Left align the first column and centre align the second
    align = c("l", "c")
  ) %>%
  kable_styling(
    # "striped" to change row colours
    # "bordered" adds lines around every cell
    bootstrap_options = c("striped", "bordered"),
    full_width = FALSE, # table only as wide as it needs to be
    position = "left" # align table to the left side of the page
  ) %>%
  # Styles the header row and colour of words
  row_spec(0, bold = TRUE, background = "#333333", color = "white") %>%
  column_spec(2, bold = TRUE, color = "black")
```

Genre	Average Score by Genre
Action	493.0569
Puzzle	496.5239
RPG	499.6938
Sports	503.3790
Strategy	500.8435

3.2.2 Findings For Average Score Per Genre

The data of average scores across the five game genres shows that Sports games have the highest average score at **503**, followed by Strategy at **501**, RPG at **500**, Puzzle at **497**, and Action at **493**.

Sports games leading in average score may reflect their competitive nature, where players are motivated to perform well and improve their rankings. Strategy games come next at **501**, which makes sense with their longer average play time, which could suggest the extra time spent on the game may contribute to stronger performance. RPG games are close behind at **500**, probably due to their progression based design where players improve their skills over time. Puzzle games sit lower at **497**, which might be due to the fact they rely more on accuracy and problem-solving than time spent playing. Whereas, Action games have the lowest average at **493**, which could be due to their fast paced and difficult gameplay, making it harder for players to frequently achieve high scores.

The gap between the highest and lowest scores is only **10 points**, showing that performance is fairly consistent across all five genres. This may be due to the platform offering a balanced scoring system where players can perform similarly no matter the type of game they choose.

3.3 Number of Sessions Per Genre

We code the total number of sessions per genre as it shows how often each genre is played on the platform, helping to identify which ones have the highest replay value among players.

```
# Assigning num_sessions_genre variable to number of sessions per game
num_sessions_genre = sessions_games %>%
  # Filters out if there are any sessions that have NA in the genre variable
  # !is.na find rows where genre is NOT missing
  filter(!is.na(genre)) %>%
  # Groups by genre so the calculation is done per genre
  group_by(genre) %>%
  # Function n() counts the number of sessions per genre
  summarise(num_of_sessions_genre = n())
```

3.3.1 Using kableExtra To Tabulate The Number of Sessions Per Genre

```
num_sessions_genre %>%
  kbl(
    # Rename the columns
    col.names = c("Genre", "Number of Sessions"),
    # Left align the first column and centre align the second
    align = c("l", "c")
  ) %>%
  kable_styling(
    # "striped" to change row colours
    # "bordered" adds lines around every cell
    bootstrap_options = c("striped", "bordered"),
    full_width = FALSE, # table only as wide as it needs to be
    position = "left" # align table to the left side of the page
  ) %>%
  # Styles the header row and colour of words
  row_spec(0, bold = TRUE, background = "#333333", color = "white") %>%
  column_spec(2, bold = TRUE, color = "black")
```

Genre	Number of Sessions
Action	3199
Puzzle	6371
RPG	2792
Sports	3995
Strategy	3643

3.3.2 Findings For Number of Sessions Per Genre

The data of the number of sessions across the five game genres shows that Puzzle games have the highest number of sessions at **6,371**, well above all other genres. This could mean that Puzzle games are the most frequently played on the platform. Their simple and accessible design likely allows players to jump in for quick sessions at any time. As each challenge feels new, it may encourage players to return more often.

Sports games have the second highest number of sessions at **3,995**, which may show their competitive appeal. The motivation for Sports game may be players wanting to keep playing to improve their performance. Strategy and Action games follow with **3,643** and **3,199** sessions, showing a fairly similar level of activity. These genres attract consistent engagement but not at the same level as Puzzle or Sports games. RPG games have the lowest number of sessions at **2,792**, which could be due to their longer gameplay. Players may return to the game less often but spend more time during each session.

Altogether, Puzzle games clearly stand out as the most frequently played genre, with more than double the sessions of RPG games. Using these insights, Play Pulse Studios should invest further in Puzzle games to build on popularity and value.

3.4 Number of Unique Players Per Genre

The number of unique players per genre shows how many different players engage with each genre, which helps to identify which ones have the widest appeal across the platform. We code using `session_games` to calculate the number of unique players as it contains the `genre` variable and the foreign key `player_id`.

```
unique_players_genre = sessions_games %>%  
  # Filters out if there are any sessions that have NA in the genre variable  
  # !is.na find rows where genre is NOT missing  
  filter(!is.na(genre)) %>%  
  # Groups by genre so the calculation is done per genre  
  group_by(genre) %>%  
  # Selects all unique player_id values in each genre and then counts how many  
  summarise(num_of_unique_players_genre = length(unique(player_id)))
```

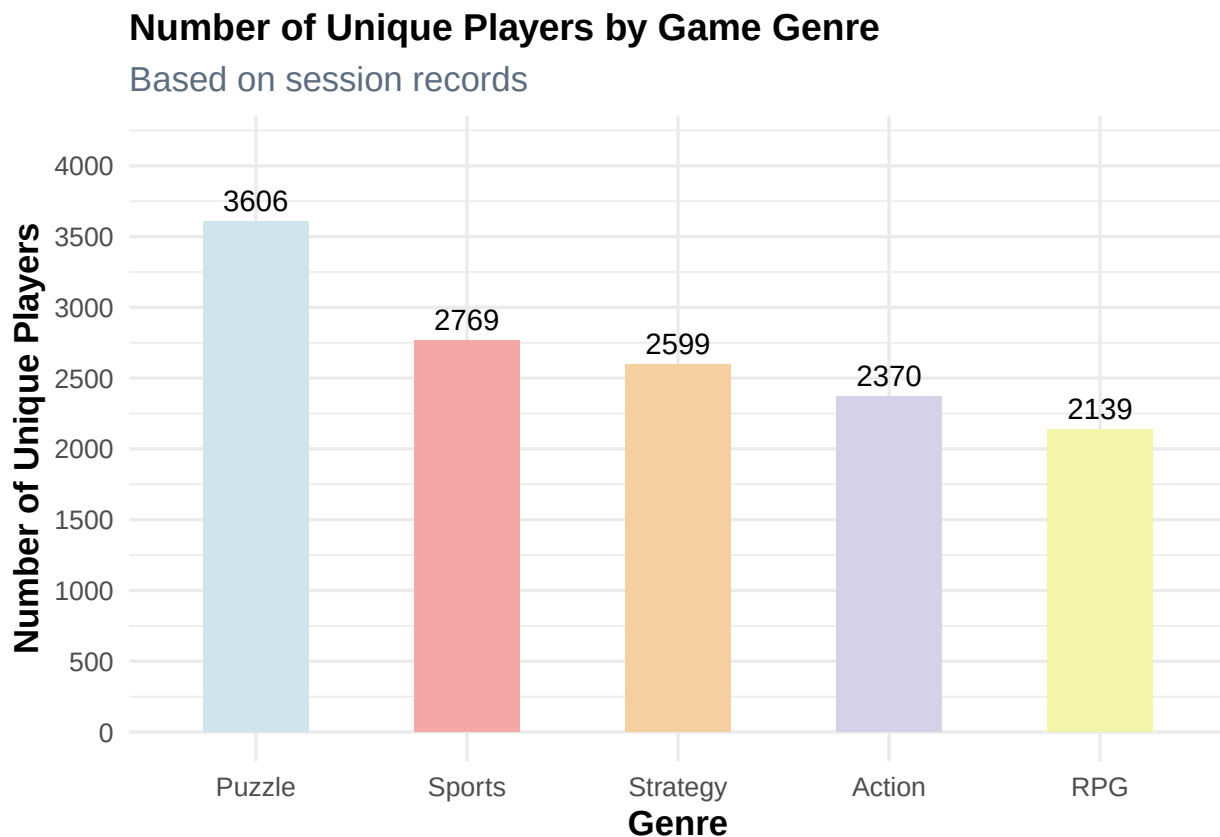
3.4.1 Plotting Number of Unique Players Per Genre

Coding a bar chart for the number of unique player per genre.

```
# Use unique_players_genre to create a bar chart  
# reorder() allows for the bars to be sorted by number of unique players  
# from highest to lowest  
ggplot(unique_players_genre,  
  aes(x = reorder(genre, -num_of_unique_players_genre),  
    y = num_of_unique_players_genre,  
    fill = genre)) +  
  
  # Draws the bars  
  geom_col(width = 0.5) +  
  
  # Labels and small adjustments  
  geom_text(  
    aes(label = num_of_unique_players_genre),  
    vjust = -0.5,  
    size = 4) +  
  
  # Assigns colours to the bar of each genre  
  scale_fill_manual(values = c(  
    "Action" = "#D5D1E9",  
    "Puzzle" = "#D0E4EE",  
    "RPG" = "#F3F5A9",  
    "Strategy" = "#F5CF9F",  
    "Sports" = "#F5A7A6")) +  
  
  # Leaves space above bars for labels  
  scale_y_continuous(  
    limits = c(0, max(unique_players_genre$num_of_unique_players_genre) * 1.15),  
    breaks = seq(0, max(unique_players_genre$num_of_unique_players_genre) * 1.15, 500)) +
```

```
# Adds titles and axis labels
labs(
  title = "Number of Unique Players by Game Genre",
  subtitle = "Based on session records",
  x = "Genre",
  y = "Number of Unique Players",) +

# Applies themes to make it look nicer
theme_minimal(base_size = 13) +
theme(
  legend.position = "none", # hides legend
  plot.title = element_text(face = "bold", size = 14),
  plot.subtitle = element_text(colour = "#5D6D7E"),
  axis.title = element_text(face = "bold"))
```



3.4.2 Findings For Number of Unique Players Per Genre

The data of unique players across the five game genres shows that Puzzle games reeled in the highest number of players at **3,606**, while RPG games had the lowest at **2,139**.

Puzzle games leading in unique players suggest they have the widest appeal on the platform. This matches earlier findings showing they also have the highest number of sessions, meaning they are both widely played and often revisited. They appeal to a broad range of players with their simple and accessible nature. Sports games come next with **2,769** players, showing strong popularity. Their competitive and social aspects may help attract a large and diverse audience. Strategy and Action games follow with **2,599**

and **2,370** players. These genres maintain steady interest but do not draw as many players as Puzzle or Sports games. RPG games have the lowest number of players at **2,139**, which could be due to their niche appeal. These games often require a large commitment, which can limit their audience.

Looking at the data as a whole, Puzzle games stand out as the most popular and engaging genre. This could suggest a strong opportunity for Play Pulse Studios to expand their Puzzle games to grow further player engagement.

4 Top Players and Their Behaviour

This task analyses the gaming behaviour of the top 10 players on the Play Pulse Studios platform. Players are ranked by total play time as a measure of overall engagement. Key metrics such as number of sessions, average play time, and average score are compared, with a box plot used to visualise score distribution.

4.0.1 Assumptions

It is assumed that total play time is a handy measure of player engagement. All recorded sessions are considered valid and reflective of real gameplay. Average score is also used as a measure of performance, and the box plot is assumed to precisely show each player's consistency and skill across their sessions.

4.1 Top 10 Players Based On Total Play Time

Here is the code using the `sessions` dataset to identify top 10 players with `play_time_minutes` variable.

```
top_10_players = sessions %>%
  # Filters out if there are any sessions that have NA in the player_id or
  # play_time_minutes variable
  # !is.na find rows where the two variables is NOT missing
  filter(!is.na(player_id), !is.na(play_time_minutes)) %>%
  # Groups by player_id so the calculation is done per player
  group_by(player_id) %>%
  # Removes any NA values then calculates the total play time per player
  summarise(total_play_time = sum(play_time_minutes, na.rm = TRUE))
) %>%
  # Sort from highest to lowest total play time
  arrange(desc(total_play_time)) %>%
  # Keep only the top 10 players
  slice(1:10)
```

4.1.1 Using kableExtra To Tabulate The Top 10 Players

```
top_10_players %>%
  # Adds a rank column from 1 to 10 before player_id
  mutate(Rank = 1:10) %>%
  # Reorders the columns so Rank appears first
  select(Rank, player_id, total_play_time) %>%
  kbl(
    # Rename the columns
    col.names = c("Rank", "Player ID", "Total Play Time (min)"),
    # Left align the first column and centre align the second
    align = c("l", "c")
  ) %>%
  kable_styling(
    # "striped" to change row colours
    # "bordered" adds lines around every cell
    bootstrap_options = c("striped", "bordered"),
    full_width = FALSE, # table only as wide as it needs to be
```



```

    position = "left" # align table to the left side of the page
  ) %>%
  # Styles the header row and colour of words
  row_spec(0, bold = TRUE, background = "#333333", color = "white") %>%
  column_spec(2, bold = TRUE, color = "black")

```

Rank	Player ID	Total Play Time (min)
1	2447	1502
2	2423	1389
3	4175	1298
4	3332	1198
5	2809	1194
6	3391	1171
7	3812	1167
8	4840	1156
9	103	1154
10	2135	1151

4.2 Total Number Of Sessions From The Top 10 Players

To code the total number of sessions from the top players we use the `sessions` dataset to calculate the total number of sessions.

```

top_10_sessions = sessions %>%
  # %in% checks if the player_id is in the top_10_players list
  filter(player_id %in% top_10_players$player_id) %>%
  # Filters out if there are any sessions that have NA in the player_id
  # !is.na find rows where the two variables is NOT missing
  filter(!is.na(player_id)) %>%
  # Groups by player_id so the calculation is done per player
  group_by(player_id) %>%
  # n() gives the total number of sessions per player
  summarise(total_sessions = n()) %>%
  # Sorts by player_id to match the order of top_10_players
  arrange(match(player_id, top_10_players$player_id))

```

4.2.1 Using kableExtra To Tabulate Total Number of Sessions Per Player

```

top_10_sessions %>%
  # Add a rank column from 1 to 10 before player_id
  mutate(Rank = 1:10) %>%
  # Reorder the columns so Rank appears first
  select(Rank, player_id, total_sessions) %>%
  kbl(
    # Rename the columns
    col.names = c("Rank", "Player ID", "Total Sessions"),
    # Centre align all columns
    align = c("c", "l", "c")
  ) %>%

```

```
kable_styling(
  bootstrap_options = c("striped", "bordered"),
  full_width = FALSE,
  position = "left"
) %>%
# Styles the header row and colour of words
row_spec(0, bold = TRUE, background = "#333333", color = "white") %>%
column_spec(2, bold = TRUE, color = "black")
```

Rank	Player ID	Total Sessions
1	2447	13
2	2423	13
3	4175	13
4	3332	9
5	2809	10
6	3391	9
7	3812	9
8	4840	10
9	103	11
10	2135	12

4.2.2 Findings for Total Number Of Sessions Per Player

The data of total sessions for the top 10 players shows that the top three players, **2447**, **2423**, and **4175**, each recorded the highest number of sessions at **13**, while players **3332**, **3391**, and **3812** had the lowest at **9 sessions** each.

Although the top three players have the same number of sessions, their total play times vary quite a lot. Player **2447** recorded **1,502 minutes**, followed by player **2423** with **1,389 minutes**, and player **4175** with **1,298 minutes**. This could suggest that player 2447 tends to spend more time in each session compared to the others. Player **2135** recorded the second highest number of sessions at **12**, showing strong engagement even though they rank lower in total play time. Whereas, Players **3332**, **3391**, and **3812** have the fewest sessions at **9**, which suggests they reach high total play time by playing longer sessions rather than playing frequently.

To summarise, the results show that the top 10 players achieve high total play time in different ways. Some players log in more often, while others play fewer but longer sessions.

4.3 Average Play Time From The Top 10 Players

The average play time of the top 10 players shows the typical session length among the most active players, coding this will help indicate whether they tend to play longer or shorter sessions compared to the overall player base.

```
top_10_avg_playtime = sessions %>%
# %in% checks if the player_id is in the top_10_players list
filter(player_id %in% top_10_players$player_id) %>%
# Filters out if there are any sessions that have NA in the play_time_minutes variable
filter(!is.na(play_time_minutes)) %>%
# Groups by player_id so the calculation is done per player
group_by(player_id) %>%
```

```
# Removes any NA values then calculates the average play time per player
summarise(tp_avg_play_time = mean(play_time_minutes, na.rm = TRUE)
)%>%
# Sort by player_id to match the order of top_10_players
arrange(match(player_id, top_10_players$player_id))
```

4.3.1 Using kableExtra To Tabulate The Top Players Average Play Time

```
top_10_avg_playtime %>%
# Adds a rank column from 1 to 10 before player_id
mutate(Rank = 1:10) %>%
# Reorders the columns so Rank appears first
select(Rank, player_id, tp_avg_play_time) %>%
kbl(
# Renames the columns
col.names = c("Rank", "Player ID", "Average Play Time (min)"),
# Centre aligns the columns
align = c("c", "l", "c")
)%>%
kable_styling(
bootstrap_options = c("striped", "bordered"),
full_width = FALSE,
position = "left"
)%>%
# Styles the header and colour of words
row_spec(0, bold = TRUE, background = "#333333", color = "white") %>%
column_spec(2, bold = TRUE, color = "black")
```

Rank	Player ID	Average Play Time (min)
1	2447	115.53846
2	2423	106.84615
3	4175	99.84615
4	3332	133.11111
5	2809	119.40000
6	3391	130.11111
7	3812	129.66667
8	4840	115.60000
9	103	104.90909
10	2135	95.91667

4.3.2 Findings For The Top 10 Players Average Play Time

The data of average play time for the top 10 players shows that player **3332** has the highest average at **133 minutes per session**, while player **2135** has the lowest at **95.9 minutes**.

Player 3332's high average is notable because they only recorded **9 sessions** yet were able to rank 4th in total play time with **1,198 minutes**. This confirms that their high total comes from longer sessions rather than total minutes played. Players **3391** and **3812** follow with **130 minutes per session**, and they also only recorded **9 sessions** each. This reinforces the idea that some top players may prefer fewer but longer sessions.

In the middle range, player **2809** averages **119 minutes**, while players **2447** and **4840** both average **116 minutes** per session, showing a balanced approach between session length. Whereas, player **2135** has the lowest average at **95.9 minutes**, but still reaches a high total play time with **12 sessions**. This suggests a different approach, relying on more frequent but perhaps shorter sessions.

Altogether, the results show two clear playing styles among the top 10 players. Some prefer fewer, longer sessions, whereas others play more often for shorter periods, yet both ways lead to high total play time.

4.4 Average Score For The Top Players

The average score of the top 10 players shows their typical performance, which helps to indicate whether higher play time is linked to better scoring on the platform. We code the average score by using the `sessions` dataset.

```
top_10_avg_score = sessions %>%
  # %in% checks if the player_id is in the top_10_players list
  filter(player_id %in% top_10_players$player_id) %>%
  # Filters out if there are any sessions that have NA in the score variable
  filter(!is.na(score)) %>%
  # Groups by player_id so the calculation is done per player
  group_by(player_id) %>%
  # Removes any NA values then calculates the average score per player
  summarise(avg_score_player = mean(score, na.rm = TRUE)) %>%
  # Sort by player_id to match the order of top_10_players
  arrange(match(player_id, top_10_players$player_id))
```

4.4.1 Using kableExtra To Tabulate The Average Score For The Top Ten Players

```
top_10_avg_score %>%
  # Adds a rank column from 1 to 10 before player_id
  mutate(Rank = 1:10) %>%
  # Reorders the columns so Rank appears first
  select(Rank, player_id, avg_score_player) %>%
  kbl(
    # Renames the columns
    col.names = c("Rank", "player ID", "Average Score"),
    # Centre aligns the columns
    align = c("c", "l", "c")
  ) %>%
  kable_styling(
    # "striped" to change row colours
    # "bordered" adds lines around every cell
    bootstrap_options = c("striped", "bordered"),
    full_width = FALSE, # table is only as wide as it needs to be
    position = "left" # aligns table to the left side of page
  ) %>%
  # Styles the header and colour of words
  row_spec(0, bold = TRUE, background = "#333333", color = "white") %>%
  column_spec(2, bold = TRUE, color = "black")
```

Rank	player ID	Average Score
1	2447	617.6923
2	2423	452.4615
3	4175	504.3077
4	3332	528.0000
5	2809	418.5000
6	3391	535.1111
7	3812	429.1111
8	4840	433.5000
9	103	438.1818
10	2135	555.8333

4.4.2 Findings For The Average Score For The Top Ten Players

The data of average scores for the top 10 players shows that player **2447** has the highest average score at **618**, while player **2809** has the lowest at **418**.

Player 2447 stands out as they also rank first in total play time, which could mean they are both highly active and skilled. In contrast, player **2135** has the second highest score at **556** despite having lower total play time, showing strong performance without as much time invested. Players **3391** and **3332** follow with good scores and longer session times, while players **2809** and **3812** have lower scores despite spending a lot of time on the platform.

These results show that time spent playing does not necessarily lead to higher scores. Some players rely on experience and consistency, while others achieve better results with less play time.

4.5 Plotting The Score Distribution For The Top Players

Now to code a box plot for the score distribution for the top 10 players.

```
top_10_boxplot = sessions %>%
  # %in% checks if the player_id is in the top_10_players list
  filter(player_id %in% top_10_players$player_id) %>%
  # Filters out any NA values in the score variable
  filter(!is.na(score)) %>%
  # Converts player_id to a factor so that ggplot can treat it as a number
  mutate(player_id = as.factor(player_id))

# Creating the boxplot
ggplot(top_10_boxplot,
       aes(x = reorder(player_id, -score), # order by highest to lowest score
           y = score,
           fill = player_id)) +

# Draws the boxplot
geom_boxplot(outlier.shape = 1,
             outlier.size = 1) +

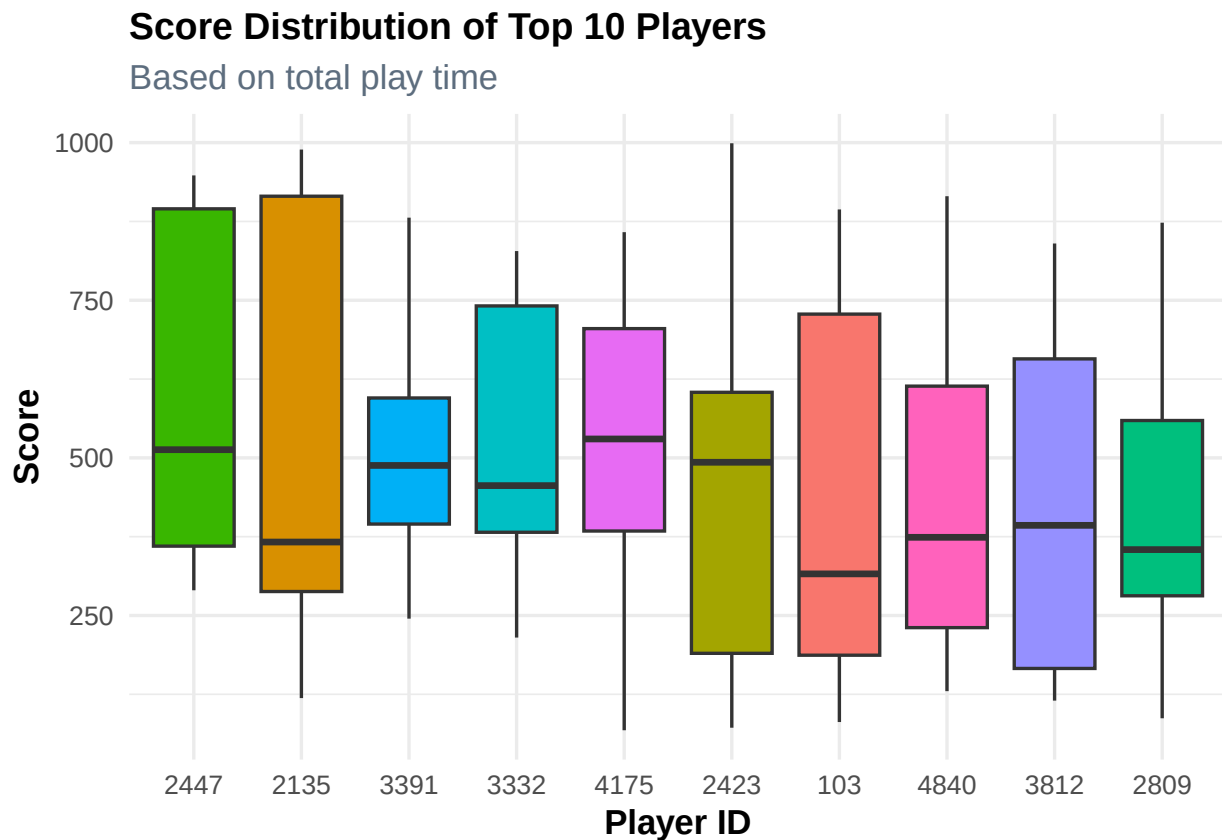
# Adds titles and axis labels
labs(
  title = "Score Distribution of Top 10 Players",
  subtitle = "Based on total play time",
```

```

x = "Player ID",
y = "Score") +

# Applies themes to make it look prettier
theme_minimal(base_size = 13) +
theme(
  legend.position = "none", # hides legend since x axis already shows IDs
  plot.title = element_text(face = "bold", size = 14),
  plot.subtitle = element_text(color = "#5D6D7E"),
  axis.title = element_text(face = "bold"))

```



4.5.1 Findings For The Score Distribution For The Top Players

The box plot shows the score distribution of the top 10 players, clearly displaying clear differences in performance and consistency.

Player **2447** has the highest median score with a narrow range, which indicates both high performance and strong consistency. This supports our earlier findings that they have the highest average score. Player **2135** shows a much wider range of scores, from very low to very high. This means they are capable of strong performance but are quite inconsistent across sessions. Players **3391** and **4175** have relatively narrow ranges in the middle, showing consistent but mediocre scoring compared to the top performers. On the other hand, player **2809** has the lowest median score and one of the widest ranges, which indicates the most inconsistency even though they have high total play time. Similarly, player **103** shows a wide range with a lower median, meaning occasional high scores but generally lower performance.

Overall, the results show that high play time does not always mean they are consistent or high scoring. Some players may combine both, while others show more variation in their performance.

5 Age Group Behaviour Comparison

This task examines whether gaming behaviour varies across different age groups on the Play Pulse Studios platform. Players are grouped into four age categories, and key metrics such as number of players, average play time, average score, and number of sessions are compared. The goal is to determine whether younger players are more active or achieve higher scores than older players.

5.0.1 Assumptions

It is assumed that the ages recorded in the dataset are accurate. The four age groups are considered suitable for segmenting the player base, with players in each group expected to showcase similar behaviour. Average play time, average score, and number of sessions are used as measures of engagement. Players with missing age data are excluded, and this is assumed to have little impact on the overall results.

5.1 Creating Age Groups

In order to analyse age group behaviours, we will run the code to group them into:

- 16 - 25
- 26 - 35
- 36 - 45
- 46+

```
players_age = players %>%
  # Filters out any NA values in the age variable
  filter(!is.na(age)) %>%
  # Creates a new variable called age_group
  # Function ifelse() checks each condition and assigns the matching group label
  mutate(age_group = ifelse(age >= 16 & age <= 25, "16-25",
                             ifelse(age >= 26 & age <= 35, "26-35",
                                     ifelse(age >= 36 & age <= 45, "36-45",
                                           "46+"))))
```

5.1.1 Using kableExtra To Tabulate The Age Groups

```
players_age %>%
  # Groups by age_group so the calculation is done per group
  group_by(age_group) %>%
  # Counts number of players in each group
  summarise(num_players = n()) %>%
  kbl(
    # Renames the columns
    col.names = c("Age Group", "Number of Players"),
    # Left align the first column and centre align the second
    align = c("l", "c")
  ) %>%
  kable_styling(
    # "striped" to change row colours
    # "bordered" adds lines around every cell
```



```
bootstrap_options = c("striped", "bordered"),
full_width = FALSE, # table only as wide as it needs to be
position = "left" # align table to the left side of the page
) %>%
# Styles the header row and colour of words
row_spec(0, bold = TRUE, background = "#333333", color = "white") %>%
column_spec(2, bold = TRUE, color = "black")
```

Age Group	Number of Players
16-25	1100
26-35	1127
36-45	1196
46+	1577

5.1.2 Findings For Age Groups

The data of the player base across the four age groups shows that the **46+ group** is the largest with **1,577 players**. This is followed by the **36 to 45 group** with **1,196 players**, the **26 to 35 group** with **1,127 players**, and the **16 to 25 group** with **1,100 players**.

Players ages 46 and above means that the platform appeals to an older audience. This could be due to the type of games offered. The 16 to 25 group has the fewest players, which could mean the platform is less appealing to younger audiences. This displays a potential opportunity for the studio to create games that target this audience.

To summarise, the platform has a diverse player base, but older age groups make up the largest share of users on the platform.

5.2 Average Play Time For Each Age Group

To be able to calculate the average play time for each age group, we must first code the `age_group` variable into the `sessions` variable.

```
# Function left_join keeps all sessions and matches player info where available
sessions_age = sessions %>%
  left_join(players_age %>% select(player_id, age_group),
            by = "player_id")

# cat() prints the value for sessions that could not be matched to a player (NA values)
cat("The NA values for sessions that matched to an age group is",
    sum(is.na(sessions_age$age_group)))
```

```
## The NA values for sessions that matched to an age group is 0
```

Here we use `sessions_age` to calculate the average play time per age group.

```
age_avg_playtime = sessions_age %>%
  # Filters out if there are any sessions that have NA in the score variable
  filter(!is.na(age_group)) %>%
  # Groups by age_group so the calculation is done per age group
  group_by(age_group) %>%
```

```
# Removes any NA values then calculates the average play time per group
summarise(avg_playtime_age = round(mean(play_time_minutes, na.rm = TRUE), 2))
```

5.2.1 Plotting The Average Play Time For Each Age Group

To visualise the average play time for each age group using a bar chart, we will use `age_avg_playtime` which was calculated from `sessions_age`.

```
ggplot(age_avg_playtime,
       aes(x = age_group, # Shows the age groups
          y = avg_playtime_age, # Shows the average play time
          fill = age_group)) + # Fills each bar with a different colour

# Draws the bars
geom_col(width = 0.5) +

# Adds value labels on top of each bar
# paste0() joins the number and "min" together
geom_text(
  aes(label = paste0(avg_playtime_age, " min")),
  vjust = -0.5, # Moves the label slightly above the bar
  fontface = "bold",
  size = 4) +

# Assigns colours to the bar of each genre
scale_fill_manual(values = c(
  "16-25" = "#FFCAE9",
  "26-35" = "#FFF0F1",
  "36-45" = "#F4DBE9",
  "46+" = "#FA9BCF")) +

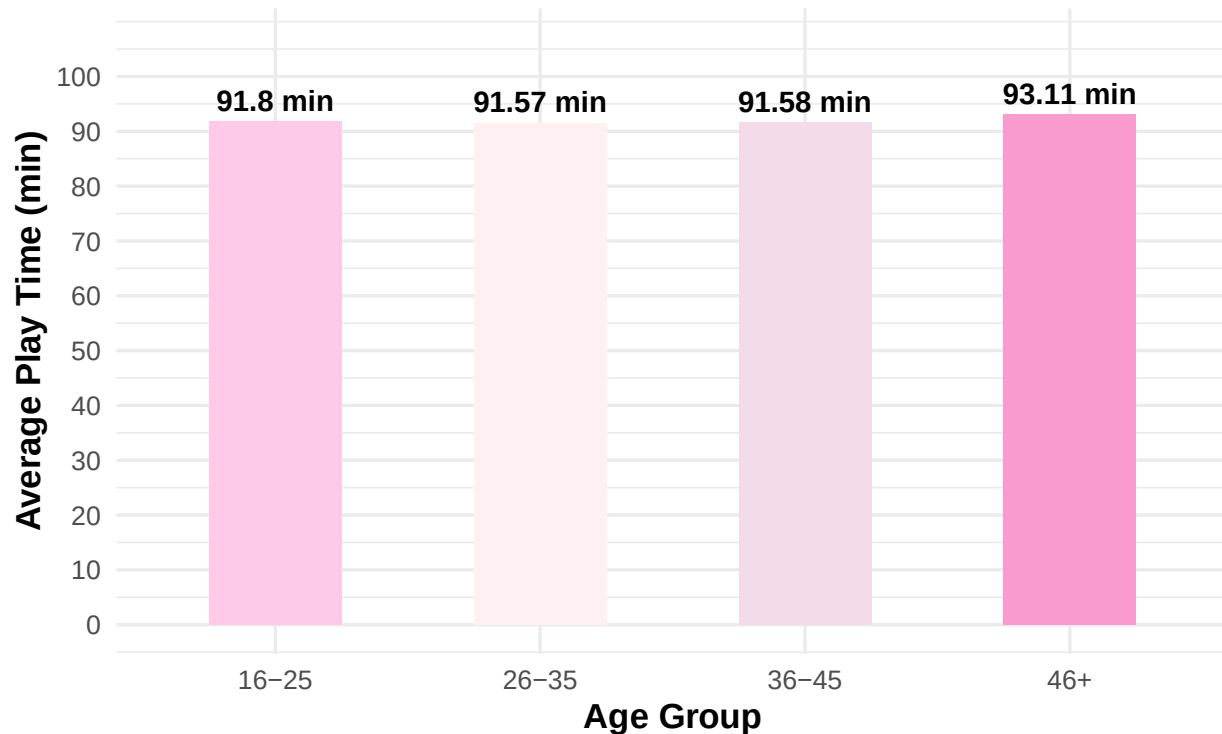
# Leaves space above bars for labels
scale_y_continuous(
  limits = c(0, max(age_avg_playtime$avg_playtime_age) * 1.15),
  breaks = seq(0, max(age_avg_playtime$avg_playtime_age) * 1.15, 10)) +

# Adds titles and axis labels
labs(
  title = "Average Play Time by Age Group",
  subtitle = "Based on player age",
  x = "Age Group",
  y = "Average Play Time (min)") +

# Applies themes to format it nicely
theme_minimal(base_size = 13) +
theme(
  legend.position = "none", # hides legend
  plot.title = element_text(face = "bold", size = 14),
  plot.subtitle = element_text(colour = "#5D6D7E"),
  axis.title = element_text(face = "bold"))
```

Average Play Time by Age Group

Based on player age



5.2.2 Findings For The Average Play Time For Each Age Group

The data of average play time across the four age groups shows that session length is very consistent, ranging from **91.6 to 93.1 minutes** per session.

The **46+ group** has the highest average at **93.1 minutes**, which could mean that older players tend to spend slightly more time per session. This may be linked to having more available free time. The **16 to 25 group** follows at **91.8 minutes**, showing that younger players are also highly engaged even though they are the smallest group. On the other hand, players aged **26 to 35** and **36 to 45** both average **91.6 minutes**, which could be due to work and family commitments.

In summary, the difference between age groups is minimal at just **1.5 minutes**, indicating that players tend to spend a similar amount of time per session regardless of age.

5.3 Average Score For Each Age Group

Using `sessions_age` we can calculate the average score per age group as it contains both the `age_group` column and `score` column.

```
age_avg_score = sessions_age %>%  
  # Filters out any sessions that have NA values in the age_group variable  
  filter(!is.na(age_group)) %>%  
  # Groups by age_group so the calculation is done per age group  
  group_by(age_group) %>%
```

```
# Removes any NA values then calculates the average score per age group
summarise(avg_score_age = mean(score, na.rm = TRUE))
```

Age Group	Average Score
16-25	496.6914
26-35	503.5342
36-45	498.0227
46+	496.7149

5.3.1 Findings For Average Score For Each Age Group

The data of average play time and average score across the four age groups show that younger players are not significantly more active or higher scoring than older players.

In terms of activity, the **46+ group** has the highest average play time at **93.1 minutes**, while the **16 to 25 group** follows at **91.8 minutes**. The **26 to 35** and **36 to 45 groups** both record the lowest at **91.6 minutes**, suggesting that younger players are not more active than older players. For scoring, the **26 to 35 group** leads with an average of **504**, followed by the **36 to 45 group** at **498**, while both the **16 to 25** and **46+ groups** average **497**. This means that middle-aged players perform slightly better, while younger players do not have an advantage.

Altogether, the differences across the four age groups are small. Player behaviour is fairly consistent regardless of age, showing that the platform appeals to and engages users across all groups.

5.4 Number of Sessions For Each Age Group

Here we use `sessions_age` to calculate the number of sessions per age group, as it contains both the `age_group` and `session_id` variables.

```
age_num_sessions = sessions_age %>%
  # Removes any sessions that have NA values in the age_group variable
  filter(!is.na(age_group)) %>%
  # Groups by age_group so the calculation is done per age group
  group_by(age_group) %>%
  # Function n() gives us the total number of session per age group
  summarise(num_sessions_age = n())
```

Age Group	Number of Sessions
16-25	4413
26-35	4538
36-45	4756
46+	6293

5.4.1 Findings For Number of Sessions For Each Age Group

The data of sessions across the four age groups shows that the **46+ group** recorded the highest number at **6,293**, well above the other groups.

This supports our earlier findings, as the **46+ group** also has the largest number of players and the highest average play time. It means that this group is the most engaged overall, both in how often they play and

how long they stay in each session. The **36 to 45 group** follows with **4,756 sessions**, then the **26 to 35 group** at **4,538**, and the **16 to 25 group** at **4,413**. This pattern shows that session frequency tends to increase with age. In contrast, the 16 to 25 group has the lowest session count and the smallest player base, in comparative to older groups making them not any more active or higher scoring.

Altogether, the results show that older players are more active on the platform. This provides an opportunity for Play Pulse Studios to create more content aimed at younger players to help increase their engagement.